

Business Requirements Document (BRD)

E-Commerce Microservices Platform

Prepared by: Rohan Khodade Version: 1.0 Date: July 2026

1. Project Overview

ShopSphere (working name — replace with your own) is a full-stack e-commerce platform built on a microservices architecture. The system allows customers to browse products, place orders, make payments, and receive order notifications, while giving admins control over product inventory and order management.

This project is built to demonstrate production-grade backend engineering practices: service decoupling, database-per-service design, centralized authentication, service discovery, and inter-service communication — the same concerns real e-commerce companies (Amazon, Flipkart, Myntra) solve at scale.

2. Stakeholders

Stakeholder	Role / Interest
Business Owner (project sponsor persona)	Wants a functional online store that can scale — new services/features added without breaking existing ones
Customer (End User)	Wants to browse products, place orders, track status, receive confirmations — smoothly and securely
Store Admin	Wants to manage product catalog, stock levels, and view/manage all orders
Development Team (you)	Wants clean service boundaries, maintainable code, and a system that showcases microservices competency to recruiters
Recruiter / Interviewer (implicit stakeholder)	Wants to see real architectural decision-making, not just CRUD — this shapes what gets emphasized in the README and demo

3. Business Objectives

1. Enable customers to self-serve: register, browse, order, pay — without manual intervention.
 2. Ensure each business capability (users, products, orders, payments, notifications) is independently deployable and maintainable.
 3. Centralize security so authentication logic isn't duplicated across services.
 4. Provide admins visibility and control over inventory and order fulfillment.
 5. Build a system architecture defensible in a technical interview — explaining *why*, not just *what*.
-

4. Scope

4.1 In Scope (MVP — build this)

Service	Responsibility
User-Service	Registration, login, JWT issuance, user profile (name, address, phone), role management (CUSTOMER, ADMIN)
Product-Service	Product CRUD (admin), catalog browsing (public), category management, stock/inventory count
Order-Service	Place order, view order history, order status lifecycle, validates product stock/price via Feign call to Product-Service
Payment-Service	Simulated payment processing, payment status tracking, linked to a specific order
Notification-Service	Sends notification (email or logged message) on order placement and payment confirmation
API Gateway	Single entry point, JWT verification, request routing to all services
Eureka Server	Service registry and discovery for all above services
React Frontend	Product listing, cart (client-side), checkout, login/register, order history, admin panel

4.2 Out of Scope (v1 — explicitly deferred, mention as "future roadmap")

- Cart-Service as an independent microservice (cart is client-side for now)
- Real payment gateway integration (Razorpay/Stripe) — simulated only

- Product search via Elasticsearch
- Review & Rating service
- Kafka-based async messaging (start synchronous via Feign; async is a stated v2 improvement)
- Config Server (centralized config) — each service self-configured for now
- Circuit breakers / Resilience4j

Explicitly listing these as "deferred, not forgotten" in your README signals architectural maturity to anyone reviewing the repo — it shows you scoped deliberately rather than ran out of time.

5. Functional Requirements

5.1 User-Service

- FR1: User can register with email, password, name, phone
- FR2: Password stored as a hash (BCrypt), never plaintext
- FR3: User can log in and receive a signed JWT (contains userId, role, expiry)
- FR4: User can view/update their profile and shipping address
- FR5: Admin-flagged users can access admin-only endpoints (role-based)

5.2 Product-Service

- FR6: Admin can create, update, delete products (name, description, price, category, stock quantity, image URL)
- FR7: Any user (authenticated or not) can browse/search product catalog
- FR8: Product-Service exposes an internal endpoint for stock/price lookup, consumed by Order-Service via Feign
- FR9: Stock quantity decrements when an order is successfully placed

5.3 Order-Service

- FR10: Authenticated user can place an order with one or more product line items
- FR11: Order-Service validates product availability and price via Product-Service (Feign call) before confirming
- FR12: Order has a status lifecycle: PLACED → PAYMENT_PENDING → CONFIRMED → SHIPPED → DELIVERED (or CANCELLED)
- FR13: User can view their own order history; Admin can view all orders
- FR14: Order-Service publishes/calls Notification-Service on order placement

5.4 Payment-Service

- FR15: Order triggers a payment record (simulated success/failure, e.g. random or rule-based mock)
- FR16: Payment status updates Order status accordingly (success → CONFIRMED, failure → PAYMENT_FAILED)
- FR17: User can view payment status/history tied to their orders

5.5 Notification-Service

- FR18: Sends notification (console log or actual email via JavaMailSender) on: order placed, payment success, payment failure
- FR19: Notification includes basic order summary (order ID, items, total)

5.6 API Gateway

- FR20: Routes all incoming requests to the correct downstream service based on path
- FR21: Verifies JWT signature and expiry on every request except `/auth/login` and `/auth/register`
- FR22: Forwards authenticated user identity (`X-User-Id`, `X-User-Role`) as headers to downstream services
- FR23: Rejects unauthenticated/invalid-token requests with 401 before they reach any service

5.7 Eureka Server

- FR24: All services (except Gateway, optionally) register themselves on startup
- FR25: Gateway and Feign clients resolve service instances via Eureka registry

6. Non-Functional Requirements

Category	Requirement
Security	JWT-based stateless auth; passwords hashed with BCrypt; secrets via environment variables, never hardcoded
Scalability	Each service independently deployable/scalable; stateless services behind Gateway
Data Isolation	Strict database-per-service — no service directly queries another's database

Category	Requirement
Maintainability	Each service has its own repo-folder, own pom.xml, own README section
Observability	Basic logging per service (SLF4J); Eureka dashboard for service health visibility
Deployability	Dockerized services with <code>docker-compose.yml</code> for one-command local spin-up
Documentation	Root README with architecture diagram, setup instructions, and API overview (Swagger per service)

7. Technology Stack

- **Backend:** Java 21, Spring Boot 4.1, Spring Cloud 2025.1.2, Spring Security, Spring Data JPA
- **Service Communication:** OpenFeign (sync), Eureka (discovery), Spring Cloud Gateway (routing)
- **Auth:** JWT (JJWT library)
- **Database:** PostgreSQL (one instance/schema per service)
- **Frontend:** React + Vite, Tailwind CSS
- **DevOps:** Docker, Docker Compose, GitHub Actions CI/CD
- **Documentation:** Swagger/OpenAPI per service

8. Success Criteria (Definition of Done for v1)

- A user can register, log in, browse products, place an order, and see a simulated payment complete — end to end, through the Gateway, with zero manual DB edits.
- Admin can add/edit products and view all orders.
- All services register with Eureka and are visible on its dashboard.
- Gateway correctly blocks unauthenticated requests and forwards user identity on valid ones.
- Entire system runs locally via `docker-compose up`.
- README clearly documents architecture, setup, and deliberate scope decisions.

To make this "eye-catching" beyond just working code:

- **Architecture diagram in README** (services, Gateway, Eureka, JWT flow) — this is often the first thing reviewed
- **A short demo GIF/video** of the flow: login → browse → order → payment → notification
- **Swagger docs** linked/screenshotted per service
- **A "Design Decisions" section** in README explaining *why* (e.g., "chose client-side cart over Cart-Service to avoid unnecessary service overhead for MVP scope" — shows judgment, not just execution)
- **Deployed, not just local** — even a partial cloud deployment (Render/Railway, as you did for HMS) demonstrates you can ship, not just code